

El Gato | Tic Tac Toe - Enunciado

// 06 Entrega y evaluación

06.1 Entregables requeridos

Repositorio Git

- Un repositorio con dos proyectos independientes: `gato-server/` y `gato-client/`.
- Cada proyecto es un Spring Boot independiente con su propio `pom.xml`.
- `README.md` en la raíz con: instrucciones de ejecución, puertos, cómo levantar 2 instancias del cliente, y descripción de los requerimientos extendidos.
- El código debe compilar y ejecutarse con `mvn spring-boot:run` sin configuración adicional.

Colección Postman

- Colección exportada (`.json`) con todos los endpoints de ambos microservicios.
- Una carpeta por microservicio (`gato-server` y `gato-client`).
- Variables de entorno configuradas: `{{serverUrl}}`, `{{client1Url}}`, `{{client2Url}}`.
- Ejemplos de request y response en cada endpoint, incluyendo al menos un caso de error por endpoint.
- Una carpeta adicional con el flujo completo de una partida: crear, unirse, mover hasta el fin.

06.2 Rúbrica de evaluación

CRITERIO	PUNTAJE
MS Server – lógica de juego Endpoints, validaciones en orden, tipos NO_TIME/TIMED, timeout lazy, tablero en cada respuesta, errores estructurados.	18%
MS Cliente – proxy + multi-server serverUrl por operación, GameRegistry, restricción de visibilidad, Security integrado correctamente, my-games agrupado por server.	15%
Spring Security – Basic Auth	00%

Spring Security - Basic Auth

Cliente: BCrypt, usuario del SecurityContext, registro público. Server: credenciales de MS, UserDetailsService desde DB, auto-registro al arranque funcional.

0%

Docker + Docker Compose

Dockerfile multistage en cada MS. docker-compose.yml levanta todo con un comando. Misma imagen cliente, dos contenedores con distinta config via environment.

12%

Tests — JUnit 5 + Mockito

Validador (6 reglas), detector de victoria (10 casos), fallback cliente (2 casos). mvn test pasa limpio.

12%

REST Nivel 2 + DTOs por capas + OpenAPI

Verbos y URLs correctos, códigos HTTP semánticos, DTOs *Request/*Response/*Command/*Result, Swagger funcional con @Operation/@ApiResponse.

12%

Flyway + YAML + config externalizada

Migraciones numeradas, ddl-auto: validate, solo .yaml, perfil dev, cero hardcoding, variables de entorno.

8%

Logging + Lombok

@Slf4j en todo, niveles info/warn/error correctos, cero System.out. @RequiredArgsConstructor, @Builder, sin @Autowired.

5%

Fallback y resiliencia

503/502 estructurado, timeout en RestTemplate configurado, nunca stack trace expuesto.

5%

Deploy Railway/Render

Server + 1 cliente con URL pública funcional. Environment prod en Postman. URLs en README.

3%

Requerimientos extendidos

2 features propias funcionando con mismo estándar técnico. Documentadas en README y Postman.

2%

06.3 Consejos para un buen proyecto

Diseña antes de codear

Antes de escribir una línea de Java, diseña en papel los endpoints de ambos microservicios, los estados posibles de una partida y cómo el cliente mapea gameId a serverUrl. Un diseño claro evita refactorizaciones costosas.

Empieza por el server, luego el cliente

El server es autónomo. Impleméntalo y pruébalo directo con Postman antes de construir el cliente. Luego construye el cliente y prueba el flujo completo con dos instancias corriendo simultáneamente.

Prueba los fallos a propósito

Para probar el fallback: crea una partida apuntando al server, luego detén el server e intenta hacer un movimiento desde el cliente. La respuesta debe ser un 503 limpio, nunca un stack trace.

Documenta este caso en Postman.

Prueba las dos instancias del cliente

El flujo completo debe funcionar con Postman apuntando a :8081 para las acciones del Jugador 1 y a :8082 para las del Jugador 2. Asegúrate de que las bases de datos MySQL de cada instancia sean independientes y que el `docker-compose.yml` las levante sin conflicto.

Fecha de entrega: Según el calendario publicado en el aula virtual. Se acepta entrega tardía con descuento de 1 punto por día hábil.

← 05 Extensión

⚙ Stack técnico →

